TP1 CryptoSE : prise en main d'ESP-IDF (bac à sable, non noté)

CryptoSE

Objectif et contrat de sortie

Cette séance de 2 h (TP1) n'est pas notée. Elle élimine les difficultés de poste de travail avant les deux TP notés (TP2 : benchmark cryptographique ; TP3 : mise à jour OTA attaquée puis défendue) et installe les conventions d'architecture du cours. Le poste de travail est Linux natif (Ubuntu) ou Windows avec WSL2 (section 3 bis). À la fin de la séance, chaque binôme doit avoir :

1. activé la version d'ESP-IDF fixée par le cours ;
2. configuré VS Code et rendu un ESP32-S3 accessible depuis Linux ;
3. compilé, flashé et surveillé une application native ESP-IDF ;
4. identifié les rôles de l'application, des composants, de FreeRTOS et de la configuration ;
5. collecté une première mesure reproductible de l'utilisation mémoire.

Le marqueur de réussite matériel est la ligne suivante dans le moniteur série :

`I (...) tutorial: CRYPTOSE_TUTORIAL_READY`

La version de référence est stockée dans `ESP_IDF_VERSION` à la racine du support. Ne remplacez pas silencieusement cette version par celle déjà présente sur votre machine : deux binômes doivent construire le même projet avec la même pile.

0. Préparation

L'installation d'ESP-IDF télécharge plusieurs gigaoctets.

1. récupérez et extrayez le support (ci-dessous) ;
2. installez les paquets système et ESP-IDF (section 2) ;
3. sous Windows, installez WSL2 et `usbipd-win` (section 3 bis) ;
4. lancez le diagnostic `python3 tools/preflight.py` **sans** la carte : aucune ligne ne doit être en `FAIL`.

Si l'un de ces points échoue, notez le message exact et signalez-le à l'enseignant dès le début de la séance.

Récupérer le support

Le support est l'archive `cryptose-tp1-student.zip`, déposée sur Moodle. Extrayez-la dans votre répertoire personnel Linux. Sous WSL2, extrayez-la dans le système de fichiers Linux (`~`), jamais sous `/mnt/c` : la compilation y est beaucoup plus lente et les droits de fichiers y sont incorrects.

```
mkdir -p ~/cryptose-tp1
unzip ~/Téléchargements/cryptose-tp1-student.zip -d ~/cryptose-tp1
cd ~/cryptose-tp1
```

Adaptez le chemin de l'archive à votre dossier de téléchargement. Sous WSL2, le dossier de téléchargement de Windows est visible sous `/mnt/c/Users/<nom>/Downloads/` : seule l'archive y est lue, le projet est extrait dans `~`. Dans la suite, **la racine du support** désigne `~/cryptose-tp1`.

1. Le modèle mental ESP-IDF

Une application ESP-IDF n'est ni un fichier `.ino`, ni une collection de fonctions globales. Le point d'entrée `app_main()` lance un système FreeRTOS déjà initialisé. Le code est organisé en composants. Chacun déclare ses sources, ses en-têtes publics et ses dépendances dans un `CMakeLists.txt`.

```
~/cryptose-tp1/                racine du support
|-- ESP_IDF_VERSION            version d'ESP-IDF imposée
|-- tp1.pdf                    ce sujet
|-- app/                       application flashable
|   |-- CMakeLists.txt         projet ESP-IDF et composants additionnels
|   |-- sdkconfig.defaults     configuration reproductible versionnée
|   `-- main/
|       |-- CMakeLists.txt
|       |-- Kconfig.projbuild  options de configuration du projet
|       `-- app_main.c         composition et cycle de vie
|-- components/
|   `-- cryptose_diag/         composant réutilisable et testable
|       |-- CMakeLists.txt
|       |-- include/cryptose_diag.h
|       `-- cryptose_diag.c
`-- tools/preflight.py         diagnostic du poste, sans modification
```

Séparation utilisée dans tout le cours :

- `app_main.c` **compose** les services et contrôle leur cycle de vie ;
- un composant possède une responsabilité, une interface publique minimale et ses dépendances explicites ;
- une tâche FreeRTOS représente une activité concurrente durable, pas une manière de découper arbitrairement le code ;
- `sdkconfig.defaults` contient les choix de cours ; `sdkconfig`, généré localement, n'est pas une source de vérité ;
- les identifiants de réseau, certificats et clés ne doivent jamais être placés directement dans le code source d'une application réelle.

Un ou plusieurs ESP32

Pour deux rôles différents, préférez deux applications (`device/`, `gateway/`) qui réutilisent des composants communs (`protocol/`, `crypto/`, `telemetry/`). Ne créez un binaire unique sélectionnant son rôle à l'exécution que si le produit doit réellement être provisionné de cette manière. Le TP3 utilisera un ESP32 pour le terminal et des services hôtes pour le serveur OTA et l'attaquant : cette topologie réduit le nombre de cartes sans masquer les frontières de confiance.

2. Installer et activer la version fixée

Le parcours de référence utilise VS Code et l'extension officielle ESP-IDF d'Espressif. L'environnement de développement intégré (IDE) permet de configurer le projet, de compiler, de flasher et d'observer la carte depuis une interface commune. En effet, ces actions utilisent la même chaîne ESP-IDF que les commandes du terminal, ce qui permet de conserver des traces reproductibles. Sous WSL2, lisez d'abord la section 3 bis : VS Code et ESP-IDF s'installent alors côté Linux.

Préparer les dépendances Linux

Installez VS Code, puis recherchez **ESP-IDF** dans la vue des extensions (`Ctrl+Shift+X`) et choisissez l'extension publiée par **Espressif Systems**. Les paquets système suivants préparent la chaîne de compilation sur Ubuntu ; les deux derniers servent aux tests hôtes du TP2 :

```
sudo apt install git wget curl unzip python3 python3-venv python3-pip \
  cmake ninja-build ccache flex bison gperf dfu-util \
  libffi-dev libssl-dev libusb-1.0-0 \
  build-essential python3-pytest
```

Le gestionnaire d'installation vérifie les prérequis. Si un paquet manque, installez le paquet nommé dans son diagnostic, puis relancez la vérification. Les dépendances système nécessitent les droits d'administration ; le SDK et les projets restent dans le répertoire personnel de l'étudiant.

Installer ESP-IDF avec le gestionnaire graphique

Dans la palette de commandes de VS Code (`Ctrl+Shift+P`), lancez **ESP-IDF : Open ESP-IDF Installation Manager**, puis choisissez **Custom Installation**. L'installation personnalisée permet de sélectionner la version fixée par le cours, tandis que **Easy Installation** sélectionne la dernière version stable disponible.

Choisissez la version indiquée dans `ESP_IDF_VERSION` (actuellement **v6.0.3**). Conservez le répertoire proposé, `~/.espressif`. Chaque étudiant utilise ainsi son propre répertoire personnel ; aucun chemin contenant le nom d'un autre utilisateur ne doit être recopié. Si le gestionnaire propose un choix de cibles, sélectionnez **ESP32-S3**. Une installation couvrant plusieurs cibles est également utilisable. Si plusieurs versions d'ESP-IDF sont installées, sélectionnez toujours celle du fichier `ESP_IDF_VERSION`.

Activer la version dans VS Code et dans le terminal

Ouvrez le dossier `app/` du support dans VS Code. Il contient le `CMakeLists.txt` de l'application à compiler. Lancez ensuite **ESP-IDF : Select Current ESP-IDF Version**, sélectionnez **v6.0.3**, puis exécutez **ESP-IDF : Doctor Command** et conservez son rapport.

Ouvrez **ESP-IDF : Open ESP-IDF Terminal**. Dans ce terminal, vérifiez la version :

```
idf.py --version
```

Le résultat doit correspondre exactement à la version du cours. Un terminal déjà ouvert avant un changement de version conserve son ancien environnement : ouvrez un nouveau terminal ESP-IDF après chaque changement.

Depuis un terminal Linux ordinaire, l'installation EIM fournit aussi un script d'activation. Depuis la racine du support :

```
cd ~/cryptose-tp1
course_idf_version=$(tr -d '[:space:]' < ESP_IDF_VERSION)
source "$HOME/.espressif/tools/activate_idf_v${course_idf_version}.sh"
idf.py --version
python3 tools/preflight.py
```

Dans un terminal ESP-IDF ouvert dans `app/`, le diagnostic se lance avec `python3 ../tools/preflight.py`.

Variante en ligne de commande

Une installation manuelle reste utilisable ; c'est aussi la plus directe sous WSL2. Exécutez les commandes suivantes dans un terminal Linux, depuis la racine du support, uniquement si vous n'avez pas déjà installé la version du cours avec EIM :

```
IDF_VERSION=$(cat ESP_IDF_VERSION)
mkdir -p "$HOME/esp"
git clone --branch "v${IDF_VERSION}" --recursive --depth 1 \
  https://github.com/espressif/esp-idf.git \
  "$HOME/esp/esp-idf-v${IDF_VERSION}"
"$HOME/esp/esp-idf-v${IDF_VERSION}/install.sh" esp32s3
```

Pour cette installation manuelle, chaque nouveau terminal doit charger l'environnement :

```
cd ~/cryptose-tp1
IDF_VERSION=$(cat ESP_IDF_VERSION)
source "$HOME/esp/esp-idf-v${IDF_VERSION}/export.sh"
idf.py --version
```

Lancez ensuite le diagnostic du cours, depuis la racine du support :

```
python3 tools/preflight.py
```

Interprétation :

- `PASS` : exigence satisfaite ;
- `WARN` : la partie hôte est utilisable, mais le matériel n'est pas encore prêt ;
- `FAIL` : ne poursuivez pas vers le flash ; corrigez la ligne concernée ;
- `SKIP` : vérification non applicable ou matériel pas encore demandé.

Pour conserver un rapport exploitable automatiquement :

```
python3 tools/preflight.py --json > preflight.json
```

3. Donner accès à la carte sous Linux

Branchez la carte avec un câble USB de **données** : certains câbles ne font que l'alimentation, et la carte s'allume alors sans apparaître côté système. Sur une ESP32-S3-DevKitC-1 à deux connecteurs, utilisez le connecteur **UART**,

relié au pont USB-série. Le connecteur USB natif est un autre chemin d'accès, qui doit être choisi explicitement lorsque l'exercice l'utilise.

Repérez le port série sans supposer son numéro :

```
lsusb
find /dev -maxdepth 1 \( -name 'ttyUSB*' -o -name 'ttyACM*' \) -print
ls -l /dev/serial/by-id/
```

Le pont USB-série apparaît habituellement comme `/dev/ttyUSB0`, et l'USB natif comme `/dev/ttyACM0`. Sous WSL2, `/dev/serial/by-id/` peut être absent : utilisez la liste `find`. L'identification de la carte repose ensuite sur la réponse d'`esptool`, pas uniquement sur le nom du port.

Si le port appartient au groupe `dialout`, ajoutez votre compte à ce groupe :

```
sudo usermod -aG dialout "$USER"
```

Déconnectez puis reconnectez la session utilisateur afin que VS Code hérite des nouveaux groupes. Fermer puis rouvrir un onglet de terminal ne renouvelle pas les groupes de la session graphique (sous WSL2, voir la section 3 bis). Vérifiez ensuite l'accès au port choisi, depuis la racine du support :

```
id -nG
test -r /dev/ttyUSB0 && test -w /dev/ttyUSB0 && echo "port accessible"
python3 tools/preflight.py --require-hardware --port /dev/ttyUSB0
```

Si une déconnexion immédiate de la session est impossible, l'enseignant peut autoriser temporairement ce seul compte sur ce seul port :

```
sudo setfacl -m "u:${USER}:rw" /dev/ttyUSB0
```

Cette autorisation temporaire peut disparaître lors d'une reconnexion USB. Elle ne remplace pas l'appartenance permanente à `dialout`. N'ouvrez pas le port en écriture à tous les utilisateurs et ne lancez pas VS Code avec `sudo`.

Le diagnostic exige un ESP32-S3 et relève notamment sa révision, sa PSRAM éventuelle et sa capacité flash. Le parc de référence dispose de 8 Mio de flash, comme `sdkconfig.defaults`. Si le diagnostic indique une autre capacité, ne changez pas la configuration seul : signalez la carte afin que toutes les mesures du groupe restent comparables.

3 bis. Windows : travailler dans WSL2

Sous Windows, tout le cours se déroule dans une distribution Ubuntu de WSL2. Le périphérique USB appartient d'abord à Windows ; `usbipd-win` le transmet à WSL.

Installer WSL2 et VS Code, une seule fois

Dans un PowerShell administrateur :

```
wsl --install -d Ubuntu-24.04
```

Redémarrez si Windows le demande, ouvrez **Ubuntu** et créez votre compte Linux. Dans VS Code côté Windows, installez l'extension **WSL** de Microsoft, puis ouvrez le dossier du support avec **WSL : Connect to WSL** ou, depuis Ubuntu, `code ~/cryptose-tp1/app`. Les extensions s'installent séparément pour la fenêtre WSL : installez l'extension ESP-IDF dans cette fenêtre. Les paquets système, ESP-IDF et le support vivent tous dans Ubuntu (section 2).

Préparer `usbipd-win`, une seule fois

Installez le paquet MSI signé publié sur la page https://github.com/dorssel/usbipd-win/releases/latest, puis ouvrez un **nouveau** PowerShell. Si la commande n'est pas encore dans le `PATH`, son emplacement standard est `C:\Program Files\usbipd-win\usbipd.exe` ; vous pouvez l'invoquer avec :

```
& "$env:ProgramFiles\usbipd-win\usbipd.exe" list
```

Branchez ensuite la carte sur le connecteur **UART**, puis, dans un PowerShell avec les droits d'administrateur :

```
usbipd list
usbipd bind --busid <BUSID>
```

`<BUSID>` est l'identifiant de la ligne correspondant à la carte. Le `bind` donne une autorisation persistante à ce périphérique ; ne sélectionnez pas un périphérique par simple ressemblance de numéro.

À chaque reconnexion de la carte

Dans un PowerShell normal :

```
usbipd attach --wsl --busid <BUSID>
```

Puis, dans Ubuntu, depuis la racine du support :

```
lsusb
ls -l /dev/ttyACM* /dev/ttyUSB* 2>/dev/null
python3 tools/preflight.py --require-hardware
```

Si plusieurs ports sont présents, imposez celui de la carte :

```
python3 tools/preflight.py --require-hardware --port /dev/ttyUSB0
```

Après une déconnexion physique ou un redémarrage de Windows, il faut généralement refaire `usbipd attach`, mais pas `bind`.

Groupe `dialout` sous WSL2

Après `sudo usermod -aG dialout "$USER"` dans Ubuntu, la session WSL doit être redémarrée pour que le groupe soit pris en compte. Fermez VS Code, puis, dans PowerShell :

```
wsl --shutdown
```

Rouvrez Ubuntu, vérifiez `id -nG`, puis refaites `usbipd attach`.

4. Construire, flasher, observer

Parcours graphique dans VS Code

Dans le dossier `app/` du support, utilisez la palette de commandes :

1. **ESP-IDF : Set Espressif Device Target**, puis **esp32s3** ;
2. **ESP-IDF : Select Port to Use (COM, tty, usbserial)**, puis le port identifié ;
3. **ESP-IDF : Build Your Project** ;
4. **ESP-IDF : Flash (UART) Your Project** ;
5. **ESP-IDF : Monitor Device**.

La barre d'état et la vue ESP-IDF donnent également accès à ces opérations. Le terminal de compilation contient la preuve de construction, tandis que le moniteur série contient la preuve d'exécution. Vérifiez les trois instantanés de diagnostic, puis le marqueur `CRYPTOSE_TUTORIAL_READY`. Fermez le moniteur de VS Code avant d'utiliser le port depuis un terminal : un seul programme peut l'ouvrir.

Les mêmes opérations dans le terminal

Depuis l'application du support :

```
cd ~/cryptose-tp1/app
idf.py set-target esp32s3
idf.py build
```

La première compilation est la plus longue. Relevez les trois valeurs de la synthèse finale : taille du binaire, taille de la plus petite partition applicative et espace libre. Elles formeront la première ligne de base des futurs benchmarks.

Choisissez ensuite le port découvert par le diagnostic :

```
export ESPPORT=/dev/ttyUSB0
idf.py -p "$ESPPORT" flash monitor
```

Quittez le moniteur avec `Ctrl-]`. Si la carte ne démarre pas automatiquement après le flash, appuyez une fois sur **RESET/EN**. N'utilisez **BOOT** que si l'enseignant vous demande explicitement d'entrer dans le chargeur série.

Ce que mesure le firmware

Le moniteur affiche d'abord la ligne de démarrage de l'application, puis trois instantanés du composant `cryptose_diag`, dont la forme est la suivante (les valeurs dépendent de la carte ; chaque instantané tient sur une seule ligne dans le moniteur) :

```
I (...) tutorial: stage=boot target=esp32s3
I (...) cryptose_diag: sample=0 idf=v6.0.3 cores=2 revision=... flash=8388608
    free_heap=... minimum_free_heap=... stack_high_water=... wifi=1 ble=1
```

Chaque instantané contient :

— la version ESP-IDF et la révision de la puce ;
— la taille de la flash, en octets ;
— le tas 8-bit libre et son minimum observé, en octets ;
— la marge de pile de la tâche de diagnostic, en **octets** (ESP-IDF compte en octets, contrairement au FreeRTOS standard, qui compte en mots) ;
— la présence des fonctions Wi-Fi et Bluetooth LE.

La valeur minimale du tas est souvent plus utile que la valeur instantanée : elle conserve la trace du point de pression maximal depuis le démarrage. Le TP2 reprendra ce principe afin de ne pas réduire un benchmark cryptographique à une seule durée. Consignez dans un fichier texte la version, la taille du binaire, l'espace libre de la partition, le minimum du tas et la marge de pile : ces cinq valeurs font partie du critère de sortie.

5. Micro-exercice : configuration et architecture

Repérez la définition `CRYPTOSE_DIAG_PERIOD_MS` dans `main/Kconfig.projbuild` et son utilisation dans `app_main.c`. Sans ajouter de nouvelle tâche :

1. exécutez `idf.py menuconfig`, ouvrez le menu **CryptoSE tutorial** et passez la période à 250 ms ;
2. reconstruisez, flashez et vérifiez la cadence des trois instantanés ;
3. identifiez le fichier local dans lequel `menuconfig` a enregistré ce choix ;
4. écrivez maintenant `CONFIG_CRYPTOSE_DIAG_PERIOD_MS=500` dans `sdkconfig.defaults`, reconstruisez et flashez : quelle cadence observez-vous, et pourquoi ?
5. supprimez `sdkconfig`, reconstruisez et flashez : la cadence est-elle désormais de 500 ms ?
6. expliquez en une phrase pourquoi le cours versionne `sdkconfig.defaults` mais ignore `sdkconfig`, et pourquoi le TP2 imposera un répertoire de compilation neuf (`idf.py -B <répertoire>`) pour chaque profil ;
7. vérifiez que le composant de diagnostic ne dépend toujours pas de l'application et que `app_main.c` reste le lieu de composition.

Une compilation propre doit conserver le marqueur `CRYPTOSE_TUTORIAL_READY`.

Comparez ensuite, en binôme, deux architectures pour un futur terminal communiquant :

— une tâche unique qui connecte le Wi-Fi, télécharge, vérifie et active une OTA ;
— un service réseau, un vérificateur de manifeste et un contrôleur de mise à jour, reliés par des messages possédés.

Écrivez trois phrases : quelle panne reste confinée, où vit l'état de sécurité, et quelle interface pourrait être testée sans carte ? Il n'existe pas de prime au nombre de tâches : l'objectif est de rendre les responsabilités et transitions auditables.

Critère de sortie du binôme

Avant de quitter la séance, montrez à l'enseignant :

```
[ ] diagnostic préalable sans FAIL, matériel exigé
[ ] compilation ciblée esp32s3 réussie
[ ] flash et moniteur série sur le port explicitement choisi
[ ] trois lignes cryptose_diag puis CRYPTOSE_TUTORIAL_READY
[ ] cinq valeurs consignées (version, binaire, espace libre, minimum du tas, marge de pile)
[ ] réponses aux étapes 4 à 6 du micro-exercice
[ ] trois phrases sur les deux architectures
```

Diagnostic rapide

Symptôme	Vérification utile	Action
`idf.py: command not found`	`idf.py --version`	ouvrir un terminal ESP-IDF ou charger le script d'activation de la version choisie
EIM installé, environnement absent	**ESP-IDF : Doctor Command**	sélectionner la version du cours puis ouvrir un terminal ESP-IDF

Symptôme	Vérification utile	Action
mauvaise version ESP-IDF	`cat ESP_IDF_VERSION`	activer l'installation versionnée du cours
carte alimentée, aucun port	`lsusb`	essayer un autre câble : il ne transporte peut-être que l'alimentation
port qui apparaît puis disparaît (Ubuntu)	`sudo dmesg -w` en rebranchant	si `brltty` s'empare du port : `sudo apt remove brltty`, puis rebrancher
`Permission denied` sur le port série	`id -nG` et droits du port	ajouter le compte à `dialout`, puis renouveler la session (`wsl --shutdown` sous WSL2)
aucun `/dev/ttyUSB*` ou `/dev/ttyACM*` sous WSL	`usbipd list` côté Windows	identifier, `bind` une fois, puis `attach --wsl`
port présent mais occupé	fermer les moniteurs concurrents, y compris celui de VS Code	relancer le diagnostic sur le port explicite
compilation pour la mauvaise puce	`idf.py set-target esp32s3`	refaire `set-target` ; en cas de doute, `idf.py fullclean` puis recompiler
flash différente de 8 Mio	ligne `flash` du diagnostic	ne pas modifier la configuration ; signaler la carte
attente après `Connecting...`	vérifier câble et connecteur	essayer RESET ; BOOT seulement en dernier recours guidé
reset continu après flash	conserver le journal complet	ne pas reflasher au hasard ; analyser la cause du reset
compilation très lente sous WSL2	`pwd`	déplacer le support dans `~`, hors de `/mnt/c`

Annexe : préparer le TP3 (facultatif aujourd'hui)

Le TP3 utilise `uv`, un gestionnaire d'environnements Python, pour les outils hôtes (serveur OTA, attaquant). En effet, cet environnement Python est distinct de celui qu'EIM fournit pour ESP-IDF : les dépendances du serveur OTA ne doivent pas modifier le SDK. L'installateur officiel s'exécute sans `sudo`. Téléchargez-le, consultez-le avec `less` (`q` pour quitter), puis lancez-le :

```
uv_install_script=$(mktemp)
curl -LsSf https://astral.sh/uv/install.sh -o "$uv_install_script"
less "$uv_install_script"
env UV_INSTALL_DIR="$HOME/.local/bin" sh "$uv_install_script"
export PATH="$HOME/.local/bin:$PATH"
uv --version
```

Cette procédure fixe l'emplacement utilisateur à `~/.local/bin` et rend `uv` disponible dans le terminal courant. L'installateur prépare aussi son ajout au chemin de recherche des prochains terminaux. Si `uv` est déjà installé, conservez-le et vérifiez `uv --version`. Dans le TP3, `uv sync --frozen --python 3.12` préparera l'interpréteur et les dépendances du projet à partir de son fichier de verrouillage, sans remplacer le Python du SDK.

Références principales

— Espressif, *Install ESP-IDF and Tools*, extension VS Code : https://docs.espressif.com/projects/vscode-esp-idf-extension/en/latest/installation.html
— Espressif, *ESP-IDF Installation Manager, Prerequisites* : https://docs.espressif.com/projects/idf-im-ui/en/latest/prerequisites.html
— Espressif, *ESP-IDF Programming Guide, Get Started (ESP32-S3)* : https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/get-started/index.html
— Espressif, *USB Serial/JTAG Controller Console* : https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-guides/usb-serial-jtag-console.html
— Microsoft, *Install WSL* : https://learn.microsoft.com/windows/wsl/install
— `usbipd-win`, *Connect USB devices under WSL* : https://github.com/dorssel/usbipd-win/wiki/WSL-support
— `usbipd-win`, versions et paquets MSI signés : https://github.com/dorssel/usbipd-win/releases/latest
— Astral, *Installation de uv* : https://docs.astral.sh/uv/getting-started/installation/